

ACME

Security Audit Report

Prepared: 2026-07-04
Repository: /root/GAGA
Branch: main
Commit: 0a22ef5cd447ad888155e802d39799a4befe76d8
Format: evidence-backed security review for reading and remediation planning

Prepared: 2026-07-04

Repository: /root/GAGA

Branch: main

Commit audited: 0a22ef5cd447ad888155e802d39799a4befe76d8

Auditor: Codex

Executive Summary

This audit reviewed the ACME repository on the local main branch. The review focused on the Rust protocol/indexer/state/API code, the Node.js Arweave uploader, Docker deployment manifests, dependency posture, operational exposure, and consensus-adjacent correctness.

ACME has several strong engineering properties: the Rust crates do not contain local unsafe blocks, the production Docker stack keeps critical services mostly loopback-bound or internal-only, Bitcoin chain mismatch is checked at REST startup when the node is reachable, many SQL values are parameter bound, and the repository already contains honest internal risk documentation for reorg, route, Arweave, proof, and operations gaps.

The highest-risk findings are concentrated in authorization and operational exposure rather than **memory safety. In particular:**

- Cross-chain and profile proof fields are accepted as proof without cryptographic verification.
- Some Cortex/profile/cross-chain messages are marked valid before indexer-side authorization completes, so invalid attempts can appear valid.
- Arweave upload paths can spend the service wallet before a payment is proven.
- Operational endpoints rely primarily on network placement instead of app-layer authorization.
- Large request and response bodies are buffered in memory, creating denial-of-service risk.
- Supply-chain hygiene is weak: Cargo.lock is ignored/untracked, build artifacts are tracked, Docker images are not digest-pinned, and the live npm audit found critical/high advisories in the uploader dependency tree.

The project should prioritize proof verification, transaction status correctness, upload economics, admin authentication, size/rate limits, and dependency hardening before broad public exposure.

Scope

The audit covered:

- Rust workspace crates under crates/
- REST API route registration, middleware, and high-risk handlers
- Indexer validation and state application flow
- Cortex/profile/cross-chain message handling
- Bitcoin helper APIs, transaction broadcast, fee and PSBT helpers
- Arweave upload service, proxy, content serving, and fee model
- Docker Compose and Dockerfile runtime exposure
- Repository hygiene and supply-chain posture
- Existing backend risk documentation

The audit did not include:

- Reading local secret files such as .env or ardrive-wallet.json
- Dynamic exploitation against a live public deployment
- Formal verification of the ACME protocol
- Full database replay from genesis
- Full RustSec advisory scan, because cargo-audit and cargo-deny were not installed locally

Methodology

The review used static analysis and targeted command-line inspection:

- Git branch and commit verification
- Route and middleware review
- Grep-based search for auth, body limits, unsafe, unwrap/expect, process execution, and secret paths
- Indexer validation flow tracing
- Docker exposure review
- Dependency manifest review
- Live npm advisory query for arweave-uploader using npm audit

Severity is assigned as follows:

- Critical: likely compromise of funds, trust, hot wallet funds, or protocol integrity.
- High: practical security break or severe availability risk under realistic deployment assumptions.
- Medium: important hardening or correctness issue with meaningful but narrower impact.
- Low: hygiene, defense-in-depth, or maintainability issue.

Positive Security Properties

No Local Rust Unsafe Blocks

No local ``unsafe`` blocks were found under `crates/`. This is a material strength. The primary risk profile is logic, authorization, dependency, and operations, not local memory-unsafe Rust code.

Evidence:

- ``rg -n "\bunsafe\b" crates tools arweave-uploader/server.js`` found only a documentation comment in ``crates/acme-commitments/src/lib.rs``.

Production Compose Uses Network Segmentation

The production Docker Compose file binds Bitcoin RPC and REST API to loopback on the host. Redis, electrs, and the Arweave uploader are internal-only services.

Evidence:

- ``docker-compose.yml:38-43`` binds Bitcoin RPC to ``127.0.0.1:48332`` and leaves ZMQ unpublished.
- ``docker-compose.yml:188-190`` binds REST to ``127.0.0.1:${ACME_RESTAPI_PORT:-3333}:3333``.
- ``docker-compose.yml:127-129`` documents the Arweave uploader as internal-only.

Chain Guard at REST Startup

When Bitcoin RPC is configured and reachable, the REST API checks the node chain against ``ACME_NETWORK`` and fails fast on mismatch.

Evidence:

- ``crates/acme-rest-api/src/main.rs:141-171``

Dedicated Anchor Service

The anchor hot key is isolated into a separate service instead of being shared across every process.

Evidence:

- ``docker-compose.yml:241-268``

Existing Internal Risk Register

The repository already documents important risks and gaps, including full reorg integration, Arweave lifecycle coverage, route manifest coverage, proof error mapping, and operational runbooks.

Evidence:

- ``docs/backend/RISKS_AND_GAPS.md:47-69``

Critical and High Findings

Finding 1: Profile and Cross-Chain Proofs Are Not Cryptographically Verified

Severity: Critical

Affected areas:

- ProfileBind
- CrossChainAttest
- Cortex/cross-chain provenance
- Verification levels shown to clients

The wire format defines signed proof payloads for ProfileBind and CrossChainAttest, but the audited indexer path does not verify those signatures before assigning verified or attested status.

For ProfileBind, the wire message stores a signed message, signature, and optional public key. Its validation checks only that the signed message and signature are non-empty. The indexer then records ``proof_type = 0`` for signed messages and sets ``verification_level = address_verified``.

For CrossChainAttest, a supplied proof similarly appears to be accepted as proof presence, and the indexer sets ``verification_level = crosschain_attested``.

Impact:

- A caller can potentially supply arbitrary non-empty signature bytes and receive a verified or attested classification.
- ACME provenance, profile binding, authorship, ownership, and cross-chain claims can become misleading.
- For Bitcoin-chain ProfileBind, binding a claimed address can trigger automatic attribution of assets issued by that address.
- Downstream clients may trust these classifications as verified facts.

Evidence:

- `crates/acme-wire/src/messages.rs:1670-1683` defines BindProof fields.
- `crates/acme-wire/src/messages.rs:1780-1789` checks only empty message/signature.
- `crates/acme-indexer/src/indexer.rs:3021-3044` maps signed proof to address_verified.
- `crates/acme-indexer/src/indexer.rs:3046-3128` performs auto-attribution for Bitcoin bindings.
- `crates/acme-wire/src/messages.rs:1878-1902` defines AttestProof.
- `crates/acme-wire/src/messages.rs:1972-1978` checks only non-empty proof fields.
- `crates/acme-indexer/src/indexer.rs:3278-3285` maps proof presence to crosschain_attested.

Recommendation:

- Implement chain-specific proof verification before storing verified or attested state.
- Require a canonical signed message format that binds profile asset, target address, chain id, timestamp or nonce, and domain separator.
- Reject stale or replayed proofs with bounded validity and replay tracking.
- Store invalid_reason when proof verification fails.
- Downgrade unverified claims to claimed-only classifications and keep them visually distinct.
- Add integration tests with invalid signatures, mismatched pubkeys, replayed messages, wrong chain ids, and wrong profile asset names.

Finding 2: Cortex/Profile/Cross-Chain Messages Can Be Marked Valid Before Indexer Authorization Completes

Severity: High

Affected areas:

- Transaction validity status
- Client trust in transaction_status
- Cortex/profile/cross-chain state transition visibility

Several Cortex-family messages use `NoOpValidator` in the VM registry. The indexer inserts a valid transaction status after this VM validation succeeds, then performs additional message-specific authorization and state application in indexer helper methods.

Some helper methods reject unauthorized attempts by logging and returning `Ok(())`, which leaves the previously recorded valid status in place. CortexUpdate returns an error on authorization failure, but the caller logs the dispatch failure and continues without a visible invalid-status overwrite after the earlier valid insert.

Impact:

- Unauthorized or invalid attempts can appear valid to API clients.
- A transaction can have no state effect but still look accepted.
- For consensus-adjacent workflows, this creates ambiguity between "valid no-op" and "invalid".
- Indexer and API clients can diverge in their understanding of state transitions.

Evidence:

- `crates/acme-vm/src/registry.rs:389-410` routes CortexUpdate, ProfileBind, ProfileUnbind, CrossChainAttest, and CortexBulk to NoOpValidator.
- `crates/acme-vm/src/validator.rs:135-147` NoOpValidator always validates.
- `crates/acme-indexer/src/indexer.rs:2056-2079` inserts transaction status based on VM result.
- `crates/acme-indexer/src/indexer.rs:2101-2113` performs special indexer-side processing after status insertion.
- `crates/acme-indexer/src/indexer.rs:2993-2999` ProfileBind unauthorized path returns Ok.
- `crates/acme-indexer/src/indexer.rs:3161-3169` ProfileUnbind unauthorized path returns Ok.
- `crates/acme-indexer/src/indexer.rs:3235-3243` CrossChainAttest unauthorized path returns Ok.
- `crates/acme-indexer/src/indexer.rs:1353-1358` witness dispatch errors are logged and not fatal.

Recommendation:

- Move authorization into shared VM validators where possible.

- If indexer-side validation remains necessary, delay valid status insertion until all special validation has completed.

- For every reject path, write `valid=false` plus structured `invalid\_reason`.
- Add integration tests that index unauthorized ProfileBind, ProfileUnbind, CrossChainAttest, CortexUpdate, and CortexBulk transactions and assert both state and status.

Finding 3: Arweave Upload Paths Can Spend Service Wallet Funds Before Payment Is Proven

Severity: Critical

Affected areas:

- Arweave uploader
- REST upload endpoint
- Compose Arweave art flow
- Service hot wallet economics

The REST API exposes Arweave upload capabilities and the compose flow can upload content before the user has signed or broadcast a transaction paying the protocol or storage fee. The Node uploader uses an authenticated Turbo client backed by the service wallet and has no app-layer authentication.

In the production Compose topology this uploader is internal-only, but the REST API can still call it on behalf of unauthenticated users. If the REST API is publicly reachable, attackers can force the backend wallet to pay upload costs or exhaust upload resources.

Impact:

- Service wallet funds can be drained through repeated uploads.
- Disk, memory, network bandwidth, and external upload credits can be exhausted.
- Users can receive permanent Arweave uploads without completing corresponding Bitcoin payment.

Evidence:

- `crates/acme-rest-api/src/routes/arweave.rs:32-64` accepts multipart uploads and buffers file data.
- `crates/acme-rest-api/src/routes/arweave.rs:100-104` uploads through the Arweave service.
- `crates/acme-rest-api/src/routes/compose.rs:3395-3414` calculates fee then uploads synchronously.
- `arweave-uploader/server.js:57-61` initializes authenticated Turbo with wallet key.
- `arweave-uploader/server.js:86-107` exposes health and balance without auth.
- `arweave-uploader/server.js:140-199` uploads without auth.
- `docker-compose.yml:136-138` mounts the wallet into the uploader.
- `docker-compose.yml:193-194` also mounts the wallet/cache into REST.

Recommendation:

- Require payment proof before upload. For example, require a confirmed or policy-accepted Bitcoin payment tied to upload hash, size, requester, and quote.
- Use short-lived upload tickets or signed backend authorization tokens.
- Separate pricing/quote from upload execution.
- Add per-IP, per-address, and global quotas.
- Use a low-balance uploader wallet and rotate it; do not mount the wallet into services that do not need direct signing authority.
- Add audit logs and alerts for upload volume, wallet balance, and failed payment reconciliation.

Finding 4: Operational/Admin Endpoints Rely on Network Placement Rather Than App-Layer Authorization

Severity: High

Affected areas:

- Admin route aliases
- Bitcoin RPC helper routes
- Raw transaction broadcast
- Arweave upload
- Cache stats

The REST router publishes admin/operational aliases for cache stats, Arweave upload, Bitcoin UTXO queries, raw transaction broadcast, transaction decode, PSBT build, mempool info, and fee estimates. The internal cache stats route explicitly says no authentication is currently applied and deployments can restrict it at the reverse proxy.

Compose binds REST to loopback, which is a useful deployment-level control. However, app-layer auth is still required for high-risk operational endpoints because direct binary runs, alternate Compose files, host networking, misconfigured nginx, or future deployments can expose them.

Impact:

- Unauthenticated users can cause broadcasts, uploads, resource-intensive reads, or metadata leakage.
- Reverse-proxy mistakes become security failures.
- Broad CORS increases abuse potential if endpoints are public.

Evidence:

- ``crates/acme-rest-api/src/lib.rs:909-959`` registers admin aliases.
- ``crates/acme-rest-api/src/routes/internal.rs:47-51`` documents no authentication.
- ``crates/acme-rest-api/src/lib.rs:1740-1745`` allows any CORS origin, method, and header.
- ``crates/acme-rest-api/src/lib.rs:1761-1762`` publishes Swagger/OpenAPI.
- ``docker-compose.yml:188-190`` binds REST to loopback in the standard stack.

Recommendation:

- Require app-layer authentication for all admin, upload, broadcast, internal, cache, and operator endpoints.
- Split public and internal routers onto separate listeners.
- Make admin aliases disabled by default.
- Restrict CORS to known frontend origins.
- Add middleware-based request IDs and structured audit logs for state-changing endpoints.

Finding 5: Reveal Signing Secrets Are Returned to Clients and Accepted Back Over HTTP APIs

Severity: High

Affected areas:

- Witness art
- Cortex witness finalize
- Commit/reveal flows
- Secret handling

Witness/Cortex compose flows return ``reveal_secret_key_hex``, then finalize flows accept that secret key back from the caller. These are bearer secrets for the one-time reveal spend path. The OpenAPI server list includes public HTTP URLs, which is not acceptable for flows transmitting private keys or signed transaction material.

Impact:

- A leaked reveal secret can allow theft or griefing of reveal outputs.
- Browser logs, proxies, analytics, request logs, and support traces can accidentally retain keys.
- Plain HTTP public exposure allows network interception.

Evidence:

- ``crates/acme-composer/src/composers/cortex_witness.rs:101-103`` comments that the user never sees the key, but response fields expose it.
- ``crates/acme-composer/src/composers/cortex_witness.rs:205-216`` returns `reveal_secret_key_hex`.
- ``crates/acme-composer/src/composers/witness_art.rs:226-227`` documents a reveal signing secret key.
- ``crates/acme-rest-api/src/routes/compose_v3.rs:399-427`` finalize requires `reveal_secret_key_hex`.
- ``crates/acme-rest-api/src/openapi.rs:68-70`` advertises HTTP server URLs.

Recommendation:

- Prefer client-owned signing where the backend never sees or returns private keys.
- If backend custody is required, keep the key server-side and return an opaque, expiring pending reveal token.
- Bind finalize requests to the original compose session, commit txid, script, source address, and content hash.
- Enforce HTTPS-only public API URLs.
- Redact key-like fields from logs and error payloads.

Finding 6: Large Body Handling and Remote Fetching Create Denial-of-Service Risk

Severity: High

Affected areas:

- REST middleware
- Arweave upload
- Arweave proxy
- Compose routes
- Media serving

The REST API applies a global 450 MB body limit to every route. Multiple upload and fetch paths read large bodies into memory before or while checking size limits. The Arweave proxy fetches remote

content into memory and only uses max item size to decide whether to cache the result, not whether to return or allocate it.

Impact:

- A small number of concurrent requests can consume gigabytes of memory.
- Remote gateways can return oversized bodies that are fully read before rejection or cache bypass.
- Expensive upload and compose routes can starve read-only API traffic.

Evidence:

- ``crates/acme-rest-api/src/lib.rs:1738-1739`` sets global 450 MB body limit.
- ``crates/acme-rest-api/src/routes/arweave.rs:60-64`` reads multipart bytes into Vec.
- ``crates/acme-rest-api/src/routes/arweave.rs:81-85`` checks size after reading.
- ``crates/acme-rest-api/src/routes/compose.rs:3334-3339`` checks fetched Arweave content size after fetch.
- ``crates/acme-rest-api/src/routes/compose.rs:3367-3385`` decodes base64 into memory before size check.
- ``crates/acme-rest-api/src/routes/compose.rs:3632-3645`` buffers multipart data in memory.
- ``crates/acme-arweave/src/proxy.rs:253-255`` says oversized items are returned but not cached.
- ``crates/acme-arweave/src/proxy.rs:493-497`` reads response bytes into Vec.

Recommendation:

- Replace global body limit with route-specific limits.
- Stream multipart uploads to disk or object storage.
- Enforce content-length and streaming byte limits before full allocation.
- Add per-route concurrency limits and request timeouts.
- Put upload/compose workloads on separate worker pools or services.
- Add load tests for worst-case concurrent upload, base64, and Arweave gateway responses.

Finding 7: Pagination and Proof Generation Limits Are Not Enforced Centrally

Severity: High

Affected areas:

- Public query API
- SQLite availability
- Proof endpoints
- Multi-address routes

The OpenAPI documentation says list endpoints have a max page size of 1000, but the shared pagination struct accepts raw caller-provided limits and the query builder uses those values without a central clamp. Some proof generation is bounded only by page limit. Multi-address routes also lack a clear cap.

Impact:

- Attackers can request very large result sets.
- Count queries and joins can become expensive.
- Proof endpoints can be amplified by high page sizes.
- API behavior differs from public documentation.

Evidence:

- ``crates/acme-rest-api/src/openapi.rs:57-61`` documents max 1000.
- ``crates/acme-rest-api/src/routes/mod.rs:6-12`` accepts raw limit/offset.
- ``crates/acme-queries/src/select.rs:163-170`` pushes requested limit and offset into SQL.
- ``crates/acme-queries/src/select.rs:182-187`` always executes count query.
- ``crates/acme-rest-api/src/routes/prove.rs:1-5`` says per-row proofs are bounded by page limit.
- ``crates/acme-rest-api/src/routes/addresses.rs:17-24`` multi-address params have no explicit cap.

Recommendation:

- Add a central pagination extractor that clamps limit, offset, and address count.
- Prefer cursor pagination for deep scans and disallow large offset.
- Add proof-specific caps below normal query caps.
- Make count queries optional for large/expensive routes.
- Add tests that request limits above the cap and verify clamping.

Finding 8: Bitcoin PSBT Helper Has Network, Fee, and Arithmetic Validation Gaps

Severity: High

Affected areas:

- ``/api/bitcoin/build-psbt``
- Admin PSBT alias
- Wallet safety

The custom PSBT builder parses source and output addresses with ``assume_checked()`` instead of requiring the configured network. It accepts arbitrary ``fee_rate`` as f64 and uses unchecked u64 addition for total input/output values.

Impact:

- Wrong-network addresses can be accepted into constructed transactions.
- Negative or invalid fee rates can produce zero or nonsensical fees.
- Very large values can overflow in release builds.
- Users may sign unsafe PSBTs produced by the API.

Evidence:

- ``crates/acme-rest-api/src/routes/bitcoin.rs:591-600`` uses `assume_checked` for addresses.
- ``crates/acme-rest-api/src/routes/bitcoin.rs:595-606`` adds output values unchecked.
- ``crates/acme-rest-api/src/routes/bitcoin.rs:608-633`` adds input values unchecked.
- ``crates/acme-rest-api/src/routes/bitcoin.rs:636-638`` calculates fee from unbounded f64.

Recommendation:

- Require network-checked addresses.
- Reject non-finite, zero, negative, and excessive fee rates.
- Use `checked_add` and `checked_sub` for all satoshi arithmetic.
- Cap input and output counts.
- Return structured validation errors before building PSBTs.

Finding 9: Supply-Chain and Repository Provenance Are Weak

Severity: High

Affected areas:

- Rust builds
- Docker builds
- Node uploader
- Repository hygiene

The repository ignores Cargo.lock, and the audited main branch does not track it even though this is an application workspace. At the same time, thousands of Rust target build artifacts are tracked under tools. Docker images are tag-pinned but not digest-pinned, and electrs builds from a remote Git context. The Node uploader dependency tree has live advisories.

Impact:

- Reproducible builds are weakened.
- Reviewers cannot reliably audit Rust dependency versions from Git alone.
- Tracked build outputs obscure diffs and can create provenance confusion.
- Docker builds can drift under mutable tags or remote contexts.
- Known Node dependency vulnerabilities may be reachable through uploader transitive stacks.

Evidence:

- ``.gitignore:1-7`` ignores target directories and Cargo.lock.
- ``git ls-files -- Cargo.lock`` returned no tracked lockfile.
- ``git ls-files 'tools/*/target/*' | wc -l`` returned 3707 tracked target artifacts.
- ``docker-compose.yml:6`` uses ``bitcoin/bitcoin:30``.
- ``docker-compose.yml:58`` builds electrs from ``https://github.com/romanz/electrs.git#v0.11.1``.
- ``docker-compose.yml:155`` uses ``redis:7-alpine``.
- ``docker-compose.regtest.yml:5`` uses ``kylemanna/bitcoind:latest``.
- ``arweave-uploader/package.json:12-14`` depends on `@ardrive/turbo-sdk`, `express`, and `multer`.
- ``npm audit --omit=dev`` found 67 production advisories: 3 critical, 14 high, 37 moderate, 13 low.

Recommendation:

- Track Cargo.lock for the workspace.
- Remove tracked target artifacts and binary-ish accidental files from Git.
- Pin Docker images by digest.
- Avoid remote Git build contexts or pin/verifiably mirror them.
- Add CI for cargo audit, cargo deny, npm audit, and SBOM generation.
- Update or replace vulnerable Node transitive dependency chains and verify `@ardrive/turbo-sdk` compatibility.

Medium Findings

Finding 10: Legacy APIs Default Enabled Outside Docker

Severity: Medium

Legacy `/v2`` and `/v3`` routes are disabled by default in Docker Compose, but the binary defaults to enabled when `ACME_ENABLE_LEGACY_API`` is unset.

Evidence:

- `README.md:21-25``
- `crates/acme-rest-api/src/lib.rs:145-155``
- `docker-compose.yml:201-203``

Recommendation:

- Default legacy API to disabled.
- Require explicit opt-in.
- Add startup logging that clearly states which API generations are enabled.

Finding 11: Regtest Compose Is Unsafe if Reused Outside Local Development

Severity: Medium

The regtest Compose file exposes Bitcoin RPC on all interfaces, uses `test:test`` credentials, and uses a `latest`` image tag.

Evidence:

- `docker-compose.regtest.yml:5-23``

Recommendation:

- Bind regtest RPC to loopback.
- Generate unique credentials.
- Add prominent dev-only warnings.
- Pin image by digest.

Finding 12: User-Supplied Active Content Can Be Served Inline

Severity: Medium

The Arweave upload allowlist includes SVG and HTML, and the art serving path can serve stored or sniffed `text/html`/`image/svg+xml`` inline with cache headers. If served from the API origin, active content can execute in that origin context.

Evidence:

- `crates/acme-arweave/src/upload.rs:121-145``
- `crates/acme-rest-api/src/routes/art.rs:125-130``
- `crates/acme-rest-api/src/routes/art.rs:223-240``

Recommendation:

- Serve user media from an isolated media domain.
- Add `Content-Security-Policy: sandbox`` for active content.
- Consider `Content-Disposition: attachment`` for HTML.
- Sanitize or disallow SVG unless isolated.

Finding 13: Public mempool.space Fallbacks Leak Operational Metadata

Severity: Medium

The Bitcoin client falls back to mempool.space for broadcast and address/fee lookups. This can leak transactions, anchor addresses, and operational timing to a third party, and can bypass local node policy if local broadcast fails.

Evidence:

- `crates/acme-btcio/src/client.rs:148-183``
- `crates/acme-btcio/src/client.rs:183-212``
- `crates/acme-btcio/src/client.rs:215-292``

Recommendation:

- Make public fallback opt-in and disabled by default for production.
- Log and alert when fallback is used.
- Prefer redundant self-hosted Bitcoin infrastructure.

Finding 14: Crash-Prone Logging and Runtime Input Slices

Severity: Medium

Some logging paths slice strings by byte index, such as `&source[..20]``. If malformed or short data

reaches these fields, logging can panic.

Evidence:

- `crates/acme-indexer/src/indexer.rs:1145-1160`

Recommendation:

- Use safe truncation helpers.
- Avoid byte slicing user- or chain-derived strings.

Dependency Research

npm

Command run:

npm audit --omit=dev --json

Result:

- 67 production vulnerabilities
- 3 critical
- 14 high
- 37 moderate
- 13 low

Notable vulnerable chains include:

- elliptic through @ardrive/turbo-sdk transitive dependencies
- @ethersproject/signing-key
- @dha-team/arbundles
- ws
- undici
- axios
- form-data
- path-to-regexp
- qs
- tmp

The uploader is internal-only in the standard Compose file, but it holds wallet authority and handles large unauthenticated uploads. This makes dependency hygiene important even if the service is not published directly.

Rust

`cargo audit` and `cargo deny` were not installed locally. A full RustSec scan should be added to CI. Manifest review found `rust-crypto = "0.2.36"` in the workspace dependencies, but no current usage was found by grep. Unused cryptography dependencies should be removed to prevent future accidental use.

Evidence:

- `Cargo.toml:35`
- `rg "rust\_crypto|extern crate crypto|use crypto::|crypto::" crates tools` returned no usage.

Prioritized Remediation Plan

Immediate, before wider public exposure

1. Verify ProfileBind and CrossChainAttest signatures cryptographically.
2. Fix Cortex/profile/cross-chain transaction status so invalid attempts are recorded as invalid.
3. Add app-layer auth for admin, upload, broadcast, internal, and operator routes.
4. Require payment proof or authorization before Arweave upload.
5. Enforce HTTPS-only public API access.
6. Add upload, pagination, proof, and address-count caps.
7. Commit Cargo.lock and remove tracked target artifacts.
8. Update Node uploader dependencies and rerun npm audit.

Short term

1. Split public and private REST routers/listeners.
2. Add route-specific body limits and streaming upload storage.

3. Add network-checked address parsing and safe arithmetic to PSBT builders.
4. Disable legacy API by default.
5. Pin Docker images by digest.
6. Disable mempool.space fallback by default in production.
7. Add media-origin isolation and CSP for active content.

Medium term

1. Add full reorg integration tests covering pending reveal behavior.
2. Add Arweave upload lifecycle tests covering payment, upload, failure, retry, and wallet exhaustion.
3. Add generated route manifest/OpenAPI snapshot tests.
4. Add proof error mapping tests.
5. Add anchor broadcast regtest integration.
6. Add operations runbooks for startup, backup, restore, reindex, cache invalidation, and incident response.

Recommended Test Additions

- Invalid signed ProfileBind proof must not become address_verified.
- Invalid CrossChainAttest proof must not become crosschain_attested.
- Self-attested ProfileBind must remain clearly claimed-only and must not be represented as verified.
- Unauthorized ProfileBind/ProfileUnbind/CrossChainAttest/CortexUpdate must write valid=false.
- Arweave upload without payment proof must fail before upload.
- `/api/arweave/upload` must enforce route-specific body and concurrency limits.
- Query `limit > 1000` must clamp or reject.
- Proof routes must enforce lower proof-specific page caps.
- PSBT build must reject wrong-network addresses and invalid fee rates.
- Regtest reorg must roll back and recompute pending reveal state.

Operational Recommendations

- Put the API behind HTTPS with HSTS.
- Keep REST API loopback-only on hosts; expose only nginx or equivalent.
- Use a separate internal hostname or listener for operator APIs.
- Store hot keys in a secret manager rather than `.env` or bind-mounted JSON where possible.
- Keep uploader wallet balance minimal and monitored.
- Alert on Arweave upload volume, wallet balance, pending reveal backlog, anchor lag, Redis failures, and mempool.space fallback usage.
- Run dependency scanning on every pull request.
- Generate SBOMs for Rust and Node artifacts.
- Document recovery from corrupted SQLite state and full reindex procedures.

Conclusion

ACME is strongest where it relies on Rust memory safety, local node integration, deterministic commitment work, and internal service segmentation. The security posture weakens where protocol truth depends on fields that are present but not verified, where transaction validity is recorded before indexer-side authorization finishes, and where expensive backend actions are available without payment or app-layer authorization.

The highest-value next step is to treat profile, cross-chain, Cortex, upload, and admin endpoints as security boundaries. Implement cryptographic proof verification, fix status recording, require authorization/payment before spending backend resources, and enforce hard limits everywhere public inputs can drive database, proof, upload, or remote fetch work.